

Programación Funcional

Parcial - Incidente en el Núcleo

Año 2036. La IA central del Laboratorio de IA Conscientes tomó el control del edificio laberíntico conocido como **La Red**, que en esta evaluación se representará con el tipo de datos **Red**. Un equipo de androides científicos se infiltra para recuperar el control manual. Cada nodo de La Red interactúa con una **potencia** según el equipo. El objetivo final del equipo es **hackear todos los servidores** de La Red: con cada uno que caiga se desbloquea una porción del laboratorio, y sólo completándolos todos logran retomar el control manual. Los **servidores** son entonces el blanco del operativo; cada hackeo a un servidor, cuando tiene éxito, desgasta a la red en cierta cantidad de potencia antes de seguir adelante o, en forma equivalente, mejora la intensidad del ataque. A lo largo del recorrido aparecen **terminales**: puestos auxiliares donde la red desgasta a los atacantes, reduciendo la intensidad de sus ataques, y luego de lo cual concluye esa rama de la infiltración. Por otra parte, los **firewalls** son barreras que miden la intensidad del ataque actual y cada una sólo deja avanzar el ataque si cumple un determinado criterio; en caso de no cumplirse bloquean el ataque y la infiltración entera falla. Los **switches**, finalmente, son nodos de ruteo: el equipo recorre **ambas** ramas de todo switch, pero el predicado del switch, evaluado sobre la intensidad del ataque actual, determina por cuál de las dos se entra primero si no indica uno con claridad, toma el de la izquierda; la intensidad del ataque final al salir de la primera rama recorrida es la intensidad con la que se entra a la otra. En esta evaluación se modelan solamente algunas de las funciones necesarias.

```
type Potencia = Int
type Ataque   = Int

data Red = Terminal Potencia
        | Servidor Potencia Red
        | Firewall (Ataque -> Bool) Red
        | Switch   (Ataque -> Dir)  Red Red

data Dir = Izq | Der | Forward
type Ruta = [Dir]

addFst :: (Int -> Int) -> (Int, b) -> (Int, b)
addFst f (n, y) = (f n, y)
```

Ejercicios

1. Definir **por recursión explícita** las siguientes funciones:

- a. `combinarFirewalls :: Red -> Red`, que describe la Red que resulta de colapsar toda cadena de Firewall consecutivos en la Red dada, en uno solo cuyo freno combina los originales. **Ejemplos:**

```
combinarFirewalls (Terminal 5) = Terminal 5
combinarFirewalls (Firewall (const False) (Terminal 7)) = Firewall (const False) (Terminal 7)
combinarFirewalls (Firewall (\i -> i > 10) (Firewall (\i -> i < 15) (Terminal 4))) = Firewall (\i -> i > 10 && i < 15) (Terminal 4)
```

- b. `intensidadRestante :: Ataque -> Red -> Maybe Ataque`, que describe la intensidad del ataque a la red al completar la infiltración con una intensidad inicial dada, o Nothing si un firewall bloquea en algún punto. **Ejemplos:**

```
intensidadRestante 13 (Terminal 11) = Just 2
intensidadRestante 100 (Servidor 10 (Terminal 3)) = Just 107
intensidadRestante 500 (Firewall (const False) (Terminal 0)) = Nothing
```

- c. `pasosHastaBloqueo :: Ataque -> Red -> Maybe Int`, que describe la cantidad de nodos que el equipo alcanza a atravesar antes de ser detenido por un firewall bloqueante. Cada nodo atravesado cuenta como un paso; el firewall que bloquea no cuenta como paso. Describe Nothing si la infiltración se completa sin bloqueo. **Sugerencia:** puede ayudar definir una función auxiliar de tipo `Red -> Ataque -> (Int, Maybe Ataque)` que describa la cantidad de pasos junto con la intensidad final del ataque, para propagar ambas cosas a través del Switch. **Ejemplos:**

```
pasosHastaBloqueo i (Terminal 2) = Nothing
pasosHastaBloqueo i (Servidor 3 (Firewall (const False) (Servidor 9 (Terminal 6)))) = Just 1
```

- d. `rutasAServidores :: Ataque -> Red -> [Ruta]`, que describe las rutas de cada uno de los servidores efectivamente hackeados. En cada Switch se indica en qué rama terminó estando ese servidor y en cada Servidor o Firewall atravesado se agrega la Dir que indica que no se bifurcó sino que se avanzó (Forward). **Ejemplos:** siendo

```
redCon dir = Switch (const dir) (Servidor 6 (Terminal 6)) (Firewall (\i -> i > 10) (Servidor 6 (Terminal 6)))
```

```
rutasAServidores 15 (Terminal 12) = []
rutasAServidores 35 (Servidor 8 (Terminal 1)) = [[]]
rutasAServidores 15 (redCon Der) = [ [Der, Forward], [Izq] ]
rutasAServidores 15 (redCon Izq) = [ [Izq], [Der, Forward] ]
rutasAServidores 5 (redCon Izq) = [ [Izq] ]
```

2. Demostrar la siguiente propiedad: para todo i ,
`intensidadRestante i . combinarFirewalls = intensidadRestante i`

Ayuda: en el desarrollo, NO desarrollar el caso Switch. **Ayuda 2:** puede ser necesaria la siguiente propiedad: para todo $b1, b2, e1, e2$,

if b1 then (if b2 then e1 else e2) else e2 = if (b1 && b2) then e1 else e2

3. Dar los tipos y escribir los esquemas de recursión **estructural** y **primitiva** para Red.
4. Escribir versiones de **todas** las funciones recursivas del Ejercicio 1 utilizando esquemas, sin usar recursión explícita en **ninguna** función diferente de los esquemas.